ACME Delivery Services

Online Analytics Pipeline

Pipeline Execution Evidence

Screenshots demonstrating end-to-end pipeline operation

1. Kafka Topics — Conduktor Console

The Conduktor console confirms that all four Debezium CDC topics are active and receiving data. The order_items topic shows the highest activity with 2,150 messages and a live produce rate of 9 msg/s, reflecting continuous order generation from the pg_cron stored procedure. The orders topic shows 305 messages with 1 msg/s. Products and customers topics are stable with no active produce rate, consistent with their 2-minute cron schedule.

- finance_db.operations.customers — 10 messages, last activity 2 minutes ago
- finance_db.operations.order_items — 2,150 messages, 9 msg/s active produce rate
- finance_db.operations.orders — 305 messages, 1 msg/s active produce rate
- finance_db.operations.products — 28 messages, last activity 2 minutes ago

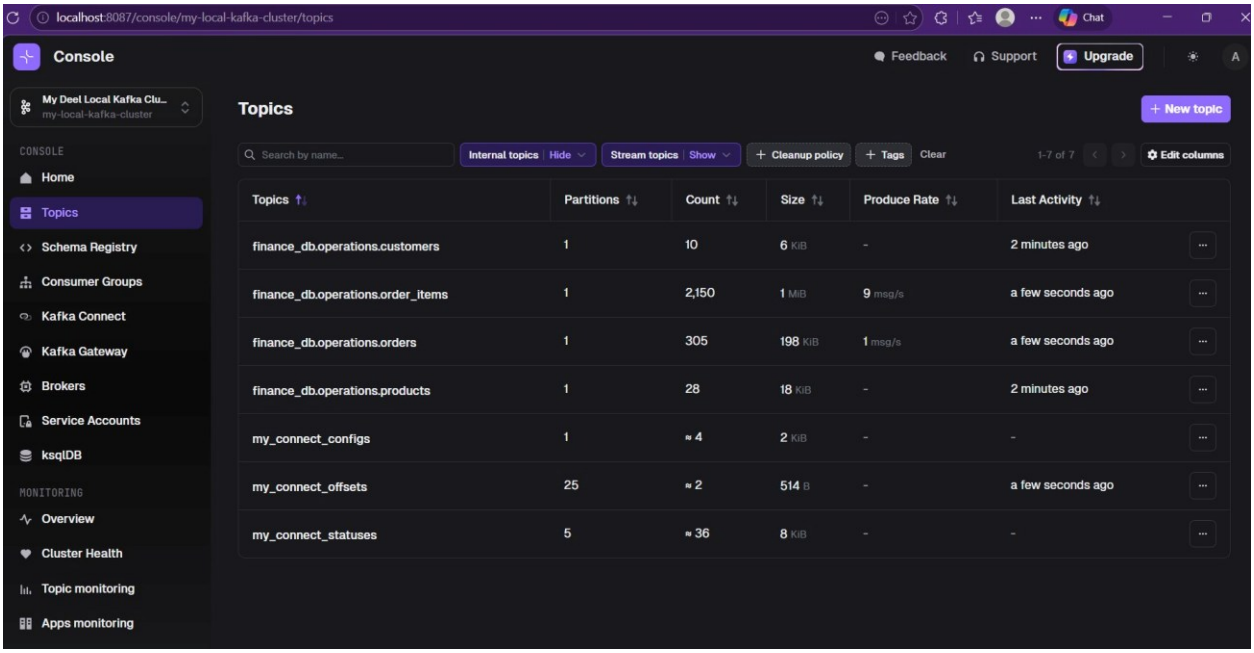

Figure 1: Conduktor console showing all four CDC topics active with message counts and produce rates

2. Dimensional Model — dim_customer (Current State)

The query SELECT * FROM analytical.dim_customer confirms that the Spark streaming pipeline is correctly upserting customer data into the analytical dimension table. 11 customers are present, each holding their latest known state. The is_active column reflects real-time changes from the source, with customer_id 2 (Computer Company) showing is_active = false, indicating a deactivation event was correctly captured and applied via the INSERT ON CONFLICT DO UPDATE pattern.

- 11 distinct customers ingested from the CDC stream
- is_active reflects latest source state — customer 2 is correctly marked inactive
- updated_at timestamps confirm data freshness from recent micro-batches

```
select * from analytical.dim_customer
```

Enter a SQL expression to filter results (use Ctrl+Space)

	customer_id	customer_name	is_active	customer_address	updated_at
1	1	Best Sellers	[v]	Blue Sky Street, 1234, Fantasy City, USA	58208-05-24 09:20:130
2	3	Care Company	[v]	Blue Sky Street, 1234, Fantasy City, USA	58208-05-23 06:26:090
3	4	Door Company	[v]	Blue Sky Street, 1234, Fantasy City, USA	58208-05-23 06:26:090
4	5	Car Sellers	[v]	Rain Forest Street, 1234, Fantasy City, USA	58208-05-24 09:20:130
5	6	Creative Company	[v]	Rain Forest Street, 1234, Fantasy City, USA	58208-05-24 09:20:130
6	7	AI Customer INC	[v]	Awesome Street, 1234, Fantasy City, USA	58208-05-24 09:20:130
7	2	Computer Company	[ ]	Blue Sky Street, 1234, Fantasy City, USA	58208-05-25 18:40:160
8	8	Ventures Company	[v]	Blue Sky Street, 1234, Fantasy City, USA	58208-05-25 18:40:160
9	9	ACME INC	[v]	Awesome Street, 1234, Fantasy City, USA	58208-05-25 18:40:160
10	10	HR Company	[v]	Awesome Street, 1234, Fantasy City, USA	58208-05-25 18:40:160
11	11	Amazing Partner LLC	[v]	Blue Sky Street, 1234, Fantasy City, USA	58208-05-25 18:40:160

Figure 2: analytical.dim_customer showing current state of all customers after CDC processing

3. SCD Type 4 — dim_customer_history

The query SELECT * FROM analytical.dim_customer_history WHERE customer_id = 1 demonstrates the SCD Type 4 history tracking. Customer 1 (Best Sellers) has two history records — history_id 1 and 6 — showing two distinct CDC events were captured. The customer_address changed between the two events (Rain Forest Street vs Blue Sky Street), proving that every state change is appended to the history table as a new row rather than overwriting the previous state.

- history_id 1: Best Sellers at Rain Forest Street (first captured state)
- history_id 6: Best Sellers at Blue Sky Street (address updated, new history row appended)
- dim_customer continues to hold only the latest state (Blue Sky Street)
- Full audit trail available via the history table for any customer_id

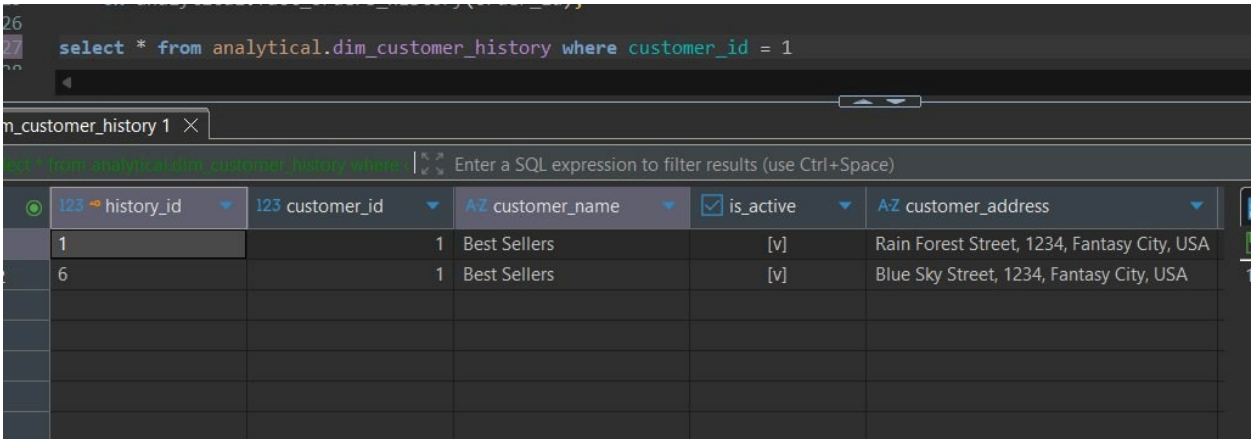

Figure 3: analytical.dim_customer_history showing two captured states for customer_id 1 — address change recorded as new history row

---

4. React Frontend — Order Analytics Dashboard

The React frontend shows the Delivery Status Report for 3/28/2026 with 110 total open orders across three non-completed statuses. Data was generated at 3:43:41 PM. The dashboard has four tabs (Delivery Status, Top Delivery Dates, Pending Items, Top Customers) each backed by a separate FastAPI endpoint querying the corresponding analytical view. The generated_at timestamp confirms near-real-time data freshness — the frontend polls every 10 seconds matching the Spark trigger interval.

- 110 total open orders on 2026-03-28
- 55 PENDING, 23 PROCESSING, 32 REPROCESSING — all non-COMPLETED statuses shown
- Generated at timestamp confirms data freshness at query time
- Four tabs navigate between the four business queries

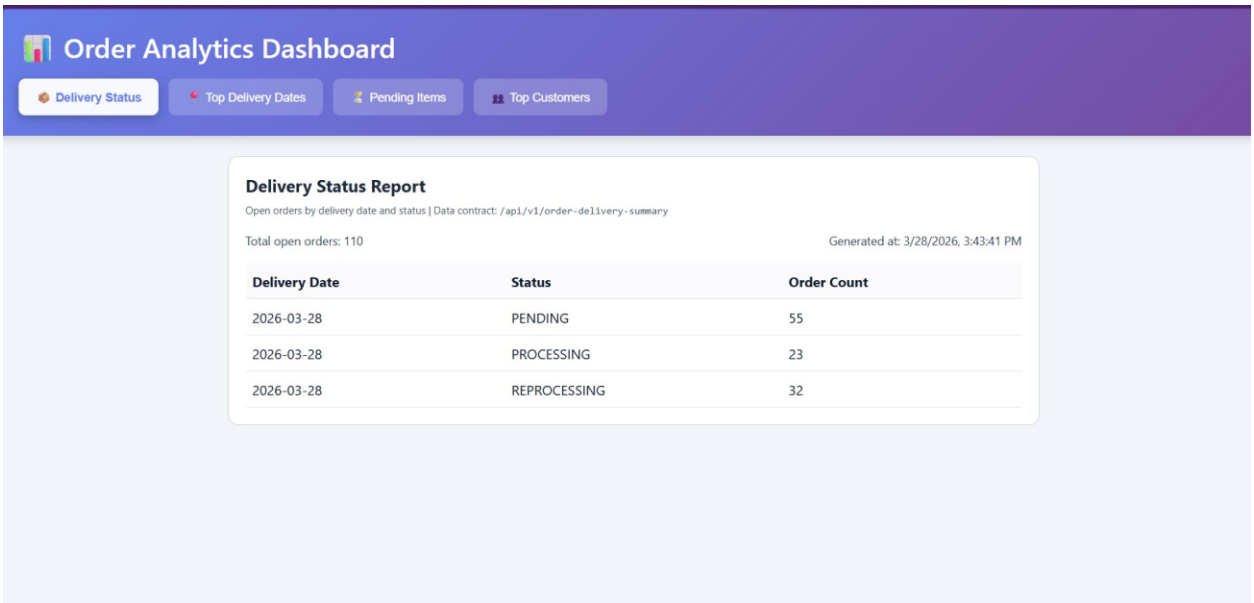

Figure 4: Order Analytics Dashboard showing Delivery Status Report with 110 open orders across PENDING, PROCESSING, and REPROCESSING statuses

---

5. Spark Consumer Logs — Pipeline Execution

The Docker container logs for spark-consumer-implementation show a complete successful micro-batch execution across all four consumers running concurrently. The logs confirm the full foreachBatch pipeline executing correctly: staging write, MERGE into target dim/fact table, history insert, and staging truncation — all completing within the same second (15:40:12) for both customers and products streams simultaneously.

- order_items: Status sync successful, staging truncated
- customers: Staging write successful -> MERGE into analytical.dim_customer -> History insert into dim_customer_history -> Staging truncated
- products: Staging write successful -> MERGE into analytical.dim_product -> History insert into dim_product_history -> Staging truncated
- All steps complete within a single micro-batch at 15:40:12, well within the 10 second trigger interval
- All four streams running concurrently as confirmed by interleaved log entries

```
spark-consumer-implementation  | 2026-03-28 15:40:11,520 INFO consumers.order_items: [order_items] Status sync successful.
spark-consumer-implementation  | 2026-03-28 15:40:11,526 INFO utils.db: Staging table analytical.fact_order_items_staging truncated.
spark-consumer-implementation  | 2026-03-28 15:40:11,526 INFO consumers.order_items: [order_items] Staging truncated.
spark-consumer-implementation  | 2026-03-28 15:40:12,918 INFO consumers.customers: [customers] Staging write successful.
spark-consumer-implementation  | 2026-03-28 15:40:12,918 INFO consumers.customers: [customers] Running MERGE into analytical.dim_customer...
spark-consumer-implementation  | 2026-03-28 15:40:12,920 INFO consumers.products: [products] Staging write successful.
spark-consumer-implementation  | 2026-03-28 15:40:12,920 INFO consumers.products: [products] Running MERGE into analytical.dim_product...
spark-consumer-implementation  | 2026-03-28 15:40:12,922 INFO utils.db: SQL executed successfully.
spark-consumer-implementation  | 2026-03-28 15:40:12,923 INFO consumers.customers: [customers] MERGE successful.
spark-consumer-implementation  | 2026-03-28 15:40:12,923 INFO consumers.customers: [customers] Inserting into history table analytical.dim_custome
r_history...
spark-consumer-implementation  | 2026-03-28 15:40:12,923 INFO utils.db: SQL executed successfully.
spark-consumer-implementation  | 2026-03-28 15:40:12,923 INFO consumers.products: [products] MERGE successful.
spark-consumer-implementation  | 2026-03-28 15:40:12,923 INFO consumers.products: [products] Inserting into history table analytical.dim_product_h
istory...
spark-consumer-implementation  | 2026-03-28 15:40:12,927 INFO utils.db: SQL executed successfully.
spark-consumer-implementation  | 2026-03-28 15:40:12,927 INFO consumers.customers: [customers] History insert successful.
spark-consumer-implementation  | 2026-03-28 15:40:12,927 INFO utils.db: SQL executed successfully.
spark-consumer-implementation  | 2026-03-28 15:40:12,927 INFO consumers.products: [products] History insert successful.
spark-consumer-implementation  | 2026-03-28 15:40:12,935 INFO utils.db: Staging table analytical.dim_customer_staging truncated.
spark-consumer-implementation  | 2026-03-28 15:40:12,936 INFO consumers.customers: [customers] Staging truncated.
spark-consumer-implementation  | 2026-03-28 15:40:12,937 INFO utils.db: Staging table analytical.dim_product_staging truncated.
spark-consumer-implementation  | 2026-03-28 15:40:12,938 INFO consumers.products: [products] Staging truncated.

v View in Docker Desktop   o View Config   w Enable Watch   d Detach
```

Figure 5: Docker logs for spark-consumer-implementation showing concurrent execution of customers and products streams with successful MERGE and history insert operations